Assignment 3: System Design & Architecture Documentation

Course	Software Development Case Study
Week	Week 6 - 7
Team Size	Maximum 2 students (same group)

Objective

Produce a complete architectural documentation package for your project: structure, data models, design decisions, and the rationale for each significant choice. The deliverable is not a diagram - it is a set of engineering artifacts that justify your architecture.

Task 1. Non-Functional Requirements & Trade-offs

Non-functional requirements (NFR) define the architecture, not the other way around. Before drawing any diagram, you must know the constraints that will drive your design decisions.

1.1. Define your NFR targets

Complete the table below for your specific project. Every cell must contain a concrete, measurable value, not a vague description.

Quality attribute	**Target (numeric)**	**Measurement method**	**Justification**
Latency	e.g. P99 < 300ms	Load testing	Explains the user experience expectation
Availability	e.g. 99.9% uptime	Uptime monitoring	Justifies replication or failover choices
Throughput	e.g. 500 RPS peak	Benchmark tool	Determines horizontal scaling needs
Consistency	e.g. Eventual / Strong	Per feature	Drives database and cache decisions
Data durability	e.g. RPO < 1 hour	DR test	Informs backup frequency and replication

1.2 Identify your primary trade-off

Every meaningful architecture requires sacrificing one property to achieve another. Choose the single most important trade-off in your system and explain it explicitly.

Trade-off axis	What you optimize FOR	What are you willing to give up
Latency vs Throughput	*[fill in]*	*[fill in]*
Consistency vs Availability	*[fill in]*	*[fill in]*
Cost vs Performance	*[fill in]*	*[fill in]*

Task 2. High-Level Architecture Diagram (C4 Model - L1 & L2)

Methodology: The C4 model (Simon Brown) provides a hierarchical way to describe system architecture at different levels of zoom. You are required to produce Level 1 (System Context) and Level 2 (Container diagram). Reference: c4model.com

Level 1 - System Context diagram

Show your system as a single box. Around it: the users who interact with it and the external systems it depends on. Arrows show the direction and purpose of communication.

- Your system (one box, named)
- All user roles (e.g. Student, Admin, Guest)
- All external systems (OAuth provider, payment API, email service, analytics)
- Arrows labeled with purpose: what data flows and why

Level 2 - Container diagram

Zoom into your system and show its major deployable units: web app, API server, database(s), message queue, cache. Each container has a technology label.

- Frontend (technology: React, Vue, etc.)
- Backend / API layer (technology: Node.js, Django, etc.)
- Database(s) - type and purpose labeled (e.g. PostgreSQL for user data, Redis for sessions)
- External services - shown as out-of-scope boxes

- Communication protocols on every arrow (REST, WebSocket, gRPC, HTTPS, TCP)

Diagram requirements

Use Draw.io, Lucidchart, or Figma. Export as PNG or PDF. Both diagrams must be embedded in the document.

- Color-code by layer (Frontend = blue, Backend = green, Database = orange, External = grey)
- Every component has a meaningful name and a one-line purpose
- Arrows indicate direction and carry a protocol label

Task 3. Architecture Decision Record (ADR)

Methodology: ADR (Michael Nygard, 2011)

An Architecture Decision Record documents a single significant architectural decision: what was decided, why, which alternatives were rejected, and the consequences. ADRs live in version control alongside code. Reference: adr.github.io

Write one ADR for the single most significant architectural decision in your project. Use the template below. Common candidates: choice of database type, monolith vs microservices, sync vs async communication, and choice of caching strategy.

ADR template (fill in each section)

ADR-001	[Give it a short descriptive title, e.g. Use PostgreSQL over MongoDB]
Status	proposed | accepted | deprecated | superseded
Date	[Date of decision]
Context	[Describe the situation. What problem required a decision? What constraints existed?]
Decision	[State the decision clearly. What exactly did you choose to do?]
Rationale	[Explain why. What properties or trade-offs led to this choice? Reference your NFRs from Task 1.]
Alternatives rejected	[List at least 2 alternatives you considered. For each: name + why rejected.]
Consequences	[What becomes easier? What becomes harder? What new risks does this introduce?]

Guidance for students without industry experience

If you have not encountered this situation professionally, reason from first principles: What does your system need most — fast reads, guaranteed consistency, horizontal scalability? Start from your NFRs (Task 1) and ask which option best satisfies them. The quality of your reasoning matters more than the choice itself.

Task 4. Detailed Component Design

For each major component (at least 4), provide a structured description using the template below. Component names must match your C4 L2 diagram from Task 2.
Draw a system architecture diagram showing all significant components and their interactions. This is the highest-level plan for your system. Make sure you can justify your architectural decisions.

Component template (repeat for each component)

Component name	[Must match the name used in your Task 2 diagram]
Purpose	What single responsibility does this component own?
Responsibilities	List 3–5 specific tasks (not descriptions of the whole system)
Inputs	Data it receives — include format and a concrete example
Outputs	Data it returns — include format, example, and error cases
Dependencies	Which other components/services does it call? Direction of calls?
NFR sensitivity	Which NFR from Task 1 most constrains this component's design?

The last field (NFR sensitivity) is new. It connects component design back to your constraints and forces explicit reasoning about why each component is designed the way it is.

Task 5. UML Diagrams

Task 5.1 - Class Diagram

Show the static structure of your system.

- Include all major domain classes (User, AuthService, Course, RecommendationEngine, etc.)
- Attributes: list key fields with data type and visibility (+ public, - private, # protected)
- Methods: include 2–3 per class, with parameters and return type

• Relationships: Inheritance, Association, Aggregation, and Composition — labeled with multiplicities

Task 5.2 - Sequence Diagram

Show dynamic interactions for 2–3 key use cases. Must include actors and system components.

- User login: User → AuthService → Database → AuthService → User
- Data submission: User → Frontend → API → Business Logic → Database → API → Frontend
- One additional scenario specific to your project
- All return messages labeled with status codes or data types

Task 5.3 - Entity-Relationship Diagram

Required if your project uses a database (which it almost certainly does).

- All entities with attributes, data types, PK and FK labeled
- Cardinality on all relationships (1:1, 1:N, N:M)
- N:M relationships resolved with a junction table
- No repeated attribute groups within a single entity

AI Tools Policy

✅What AI Tools May Be Used For

- You may use AI tools (ChatGPT, Claude, Copilot, Cursor, v0, Notion AI, etc.) to assist with:
- Generating project ideas and exploring new directions.
- Creating preliminary user stories or draft requirements to respond to and revise.
- Getting feedback on your writing clarity or structure.
- Examples (e.g., what is a good non-functional requirement).
- Recommending diagram layouts or component names to use.

🚫What AI Tools May NOT Be Used For

The following uses are strictly prohibited:

- Submitting a full or near-complete response to any section of the assignment as your own.
- Submitting AI output as the final deliverable without meaningful editing, personal judgment, and reflection.
- Substituting your analysis - requirements should be based on what you know about the problem, not on AI.

- Generating a complete or near-complete answer to any part of the assignment and submitting it as your own.
- Using AI output as the final deliverable without meaningful editing, personal judgment, and reflection.
- Replacing your own analysis - requirements must reflect your understanding of the problem, not the AI's.

🗨 Plagiarism & AI Detection

Every uploaded document will be run through an anti-plagiarism tool with an AI-generated content detection. Papers, which seem to be completely or predominantly AI-generated - without recorded application and self-reflection - will be classified as academic dishonesty, and can receive a zero mark.

The object is not to get you, but to make sure that you are the one learning.

Grading Overview

Criterion	What to demonstrate	Points
Task 1 - NFR & Trade-offs	NFRs measurable, trade-off reasoned, lecture grounded	**10**
Task 2 - C4 Diagram	L1 + L2 correct, protocols labeled, C4 conventions followed	**30**
Task 3 - ADR	Structure complete, alternatives rejected, consequences named	**10**
Task 4 - Component Design	Coverage, specificity, consistency, NFR linkage	**20**
Task 5 - UML	Class + Sequence + ER, each graded separately	**30**
TOTAL		**100**

Academic Standards & Scope

The scope of this assignment extends beyond the task instructions below. You are expected to integrate concepts from the weekly reading list and lecture notes delivered in class. Work that addresses only the explicit task requirements without drawing on course materials will be assessed at a lower level.

You are not limited to the minimum requirements described in each task. If your knowledge or experience allows you to apply a more rigorous approach, you are encouraged to do so. Briefly explain your reasoning.

Every technique or artifact you produce must be grounded in a named methodology or framework. It is not enough to deliver an output; you must apply the

correct method, use it accurately, and reference its source. If you apply a method not covered in lectures, name it, cite it, and justify why it is appropriate for the context.